

PROJET IOT :

CharioTrack

Système de Suivi de Chariot en Temps Réel

Champ	Détail
Étudiants	• Yosser AYADI • Naziha GOUBAA
Encadrants	• Mme Sirine KHALFALLAH • Mme Hadda BEN ELHADJ
Type de projet	Prototype IoT embarqué



Table des matières

1. Introduction et Contexte	4
1.1 Objectifs du projet.....	4
1.2 Technologies utilisées	4
2. Architecture Générale du Système.....	5
3. Matériel Utilisé	5
4. Mesure de la Batterie : Pont Diviseur	6
4.1 Principe du pont diviseur	6
4.2 Avantages de cette méthode	7
5. Schéma de Câblage Complet	7
5.1 Tableau de connexion détaillé	7
5.2 Circuit d'alimentation	8
6. Code Embarqué (ESP32)	8
6.1 Bibliothèques requises	8
6.2 Calcul de la position (Odométrie).....	9
6.3 Endpoints du serveur Flask	9
7. Procédure de Test — Composant par Composant.....	9
8. Résultats et Conclusion	10
8.1 Fonctionnalités réalisées	10
8.2 Difficultés rencontrées	10
8.3 Ce que ce projet nous a appris	10
9. Perspectives et Améliorations	11



Table des figures

Figure 1 : Principe du pont diviseur.....	6
Figure 2 : Circuit d'alimentation	8

Table des tableaux

Tableau 1 : Technologies utilisées.....	4
Tableau 2: Les composants.....	6
Tableau 3 : Connexion détaillé	8
Tableau 4 : Bibliothèques requises.....	9
Tableau 5 : Endpoints du serveur Flask.....	9
Tableau 6 : Le test du Composant par Composant	10

1. Introduction et Contexte

CharioTrack est notre projet IoT (Internet of Things). L'objectif est de concevoir un système embarqué complet permettant de surveiller un chariot en temps réel : ses mouvements, les obstacles devant lui, sa position et son niveau de batterie.

Ce projet combine l'électronique embarquée (ESP32), plusieurs capteurs physiques, et une application web affichant les données sur un tableau de bord interactif accessible depuis n'importe quel navigateur sur le réseau local.

1.1 Objectifs du projet

- Détecter les mouvements, l'arrêt et les chocs du chariot (MPU-6050)
- Mesurer la distance aux obstacles devant le chariot (HC-SR04)
- Calculer la position X/Y et la distance parcourue via l'odométrie (LM393 + disque codeur)
- Mesurer le niveau de la batterie Li-ion 18650 via un pont diviseur de résistances
- Stocker toutes les données dans une base SQLite via un serveur Flask
- Afficher un tableau de bord web en temps réel depuis n'importe quel navigateur

1.2 Technologies utilisées

Ce tableau regroupe les principales technologies intégrées dans le système du chariot connecté. Il présente les composants matériels et logiciels ainsi que leurs fonctions dans l'architecture globale du projet.

Couche	Technologie	Rôle
Matériel	ESP32 DevKit	Cerveau : Wi-Fi, I2C, GPIO, ADC
Matériel	MPU-6050	Accéléromètre + Gyroscope (I2C)
Matériel	HC-SR04	Capteur ultrason frontal (GPIO)
Matériel	LM393 + disque codeur	Encodeurs de roues (interruptions)
Matériel	Pile Li-ion 18650	Alimentation 3.7V → boost 5V
Matériel	R 10kΩ + R 20kΩ	Pont diviseur mesure batterie → ADC
Serveur	Python Flask	Backend REST API
Serveur	SQLite	Base de données locale
Interface	HTML+CSS+JS	Tableau de bord web temps réel

Tableau 1 : Technologies utilisées

2. Architecture Générale du Système

Le système CharioTrack est composé de trois couches distinctes :

Couche 1 — Matériel (Hardware)

L'ESP32 lit tous les capteurs : MPU-6050 (I2C), HC-SR04 frontal (GPIO + pont diviseur), 2x LM393 avec disque codeur (interruptions GPIO34/35), et la batterie Li-ion 18650 dont la tension est mesurée via un pont diviseur $R1=10k\Omega + R2=20k\Omega$ sur GPIO33.

Couche 2 — Serveur (Backend)

Un serveur Python Flask sur le PC reçoit les données JSON de l'ESP32 (HTTP POST), les stocke dans SQLite, et les sert à la page web (HTTP GET). Il expose aussi les endpoints /history, /stats, et /history/evenements.

Couche 3 — Interface Web (Frontend)

La page web (HTML + CSS + JS) interroge le serveur toutes les secondes.

Elle affiche : statut, position X/Y, distances ultrason, accéléromètre, batterie, carte de trajet interactive et journal des événements horodaté.

Flux de données complet :

Capteurs → ESP32 → Wi-Fi (HTTP POST) → Flask + SQLite → HTTP GET → Navigateur Web

3. Matériel Utilisé

Le tableau suivant détaille les différents composants électroniques utilisés lors de la réalisation du prototype. Il indique également la quantité, le rôle de chaque composant et son coût estimatif.

Composant	Qté	Rôle dans le projet	Prix (TND)
ESP32 DevKit	1	Cerveau : Wi-Fi, GPIO, I2C, ADC, interruptions	33,000
MPU-6050 (GY-521)	1	Accéléromètre 3 axes + gyroscope 3 axes — détection chocs	9,000
HC-SR04 (ultrason)	3	Mesure distance obstacles frontaux (0°)	5,000
LM393 + disque codeur noir	2	Encodeur optique : comptage ticks de rotation des roues	11,000
Pile Li-ion 18650	1	Alimentation principale 3.7V (4.2V chargée)	8,000
Porte-pile 1x18650	1	Support avec fils rouge (+) et noir (-)	2,500
Module boost 5V	1	Convertit 3.7V → 5V stable pour l'ESP32	3,800
Breadboard 400 points	1	Support ponts diviseurs + résistances	4,600
Câbles Dupont nappe arc-en-ciel	~40	Connexions modules → ESP32 et breadboard	6,000

Résistance 10 kΩ	1	R1 pont diviseur batterie	0,150
Résistance 20 kΩ	1	R2 pont diviseur batterie	0,150
LED rouge	1	Indicateur visuel alerte obstacle/choc	0,200
Résistance 220 Ω	1	Limitation courant LED	0,150
Buzzer actif	1	Alerte sonore	2,000
Câble Micro-USB	1	Programmation via Arduino IDE	8,500

Tableau 2: Les composants

4. Mesure de la Batterie : Pont Diviseur

La pile Li-ion 18650 peut atteindre 4.2V quand elle est pleinement chargée. L'ESP32 ne supporte que 3.3V maximum sur ses entrées ADC. Sans protection, on risque d'endommager la carte de façon permanente.

4.1 Principe du pont diviseur

Deux résistances R1 et R2 en série divisent la tension de la batterie avant de l'envoyer sur le GPIO33 (ADC de l'ESP32) :

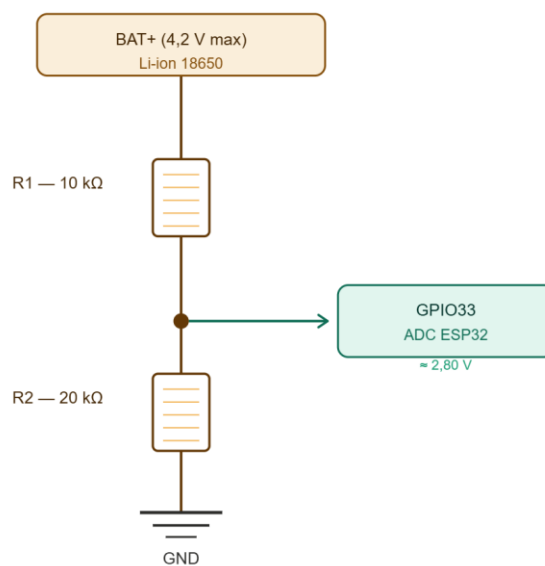


Figure 1 : Principe du pont diviseur

Calcul :

$$V_{adc} = V_{bat} \times R2 / (R1 + R2) = 4.2V \times 20 / (10 + 20) = 4.2V \times 0.667 = 2.80V \checkmark (< 3.3V)$$

4.2 Avantages de cette méthode

- Simple : seulement 2 résistances, aucun composant supplémentaire
- Économique : coût < 0.50 TND total
- Suffisamment précis pour afficher un pourcentage indicatif
- Compatible directement avec l'ADC 12 bits de l'ESP32

5. Schéma de Câblage Complet

5.1 Tableau de connexion détaillé

Le tableau suivant présente les connexions réalisées entre les différents composants électroniques et les broches de l'ESP32. Il permet de visualiser le câblage utilisé ainsi que les remarques importantes liées à l'alimentation et à la protection des entrées GPIO.

Composant	Broche composant	Broche ESP32	Fil	Remarque
MPU-6050	VCC	3.3V	Rouge FM	PAS 5V — grille à 5V !
MPU-6050	GND	GND	Noir FM	Masse commune
MPU-6050	SDA	GPIO 21	Violet FM	Bus I2C données
MPU-6050	SCL	GPIO 22	Bleu FM	Bus I2C horloge
MPU-6050	AD0	GND (direct)	Noir FM	Adresse I2C = 0x68
HC-SR04 Frontal	VCC	5V (VIN)	Rouge FM	Alim 5V
HC-SR04 Frontal	GND	GND	Noir FM	
HC-SR04 Frontal	Trig	GPIO 26	Orange FM	Signal déclenchement
HC-SR04 Frontal	Echo	R1→R2→GND (breadboard)	Bleu FM	Pont diviseur obligatoire !
HC-SR04 Frontal	Trig	GPIO 5	Orange FM	
HC-SR04 Frontal	Echo	GPIO 18 (via breadboard)	Bleu FM	Pont diviseur
HC-SR04 Frontal	Trig	GPIO 14	Orange FM	
HC-SR04 Frontal	Echo	GPIO 12 (via breadboard)	Bleu FM	Pont diviseur
LM393 Gauche	VCC	5V	Rouge FM	
LM393 Gauche	GND	GND	Noir FM	

LM393 Gauche	OUT	GPIO 34	Vert FM	Input only — pas de pullup
LM393 Droite	OUT	GPIO 35	Vert FM	Input only — pas de pullup
Bat 18650 (+)	Fil rouge	R1 (10kΩ) breadboard	Rouge MM	Pont diviseur batterie
Bat 18650 (-)	Fil noir	GND	Noir MM	Masse commune
Milieu pont bat	Nœud R1+R2	GPIO 33	Jaune MM	Lecture ADC batterie
R2 pont bat	20kΩ bas	GND	Noir MM	
Buzzer	(+)	GPIO 25	Orange FM	
LED rouge	Anode	GPIO 2 (via 220Ω)	Rouge FM	
Module boost	OUT 5V	VIN ESP32	Rouge MM	Alim principale
Module boost	GND	GND	Noir MM	

Tableau 3 : Connexion détaillé

5.2 Circuit d'alimentation

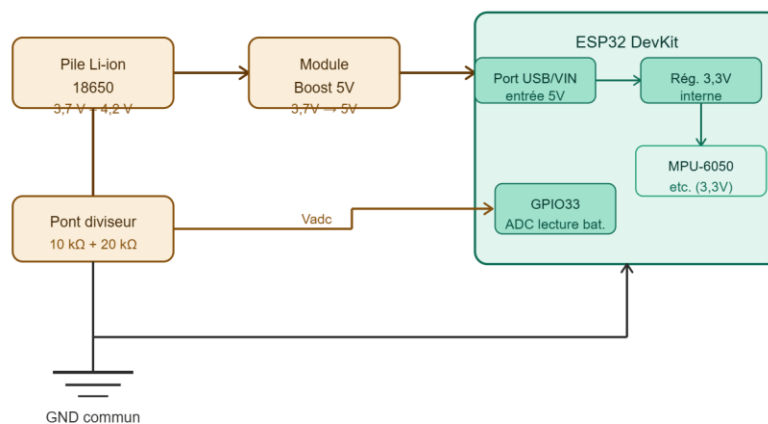


Figure 2 : Circuit d'alimentation

6. Code Embarqué (ESP32)

6.1 Bibliothèques requises

Dans ce projet, on a utilisé plusieurs bibliothèques indispensables au fonctionnement du système.



Voici le tableau qui représente les principales bibliothèques utilisées ainsi que leur rôle dans la communication, la lecture des capteurs et l'échange de données.

Bibliothèque	Source	Rôle
Wire.h	Incluse avec ESP32	Communication I2C
MPU6050.h	Electronic Cats (gestionnaire bibl.)	Lecture MPU-6050
WiFi.h	Incluse avec ESP32	Connexion Wi-Fi
HTTPClient.h	Incluse avec ESP32	Envoi HTTP POST
ArduinoJson.h	Benoit Blanchon (gestionnaire bibl.)	Création JSON

Tableau 4 : Bibliothèques requises

6.2 Calcul de la position (Odométrie)

La position X/Y est calculée sans GPS, uniquement à partir du comptage de ticks des disques codeurs :

1. Chaque fenêtre du disque codeur = 1 tick (compté par interruption GPIO)
2. Distance : $d = \text{ticks} \times (\pi \times \varnothing_{\text{roue}} / \text{ticks_par_tour})$
3. Angle : $\Delta\theta = (d_{\text{droite}} - d_{\text{gauche}}) / \text{écartement_roues}$
4. Position : $X += d_{\text{moy}} \times \cos(\theta)$, $Y += d_{\text{moy}} \times \sin(\theta)$

6.3 Endpoints du serveur Flask

Le tableau suivant présente les différents endpoints du serveur ainsi que leur utilisation dans la gestion des données.

Méthode	Endpoint	Description
GET	/data	Dernières données temps réel → page web
POST	/data	ESP32 envoie ses données → stockage SQLite
GET	/history	Historique complet des mesures
GET	/stats	Statistiques de session (chocs, distance max...)
DELETE	/history	Effacer toute la base de données

Tableau 5 : Endpoints du serveur Flask

7. Procédure de Test — Composant par Composant

Méthode : tester chaque composant séparément avant l'assemblage final. Pour chaque test : copier le code → coller dans Arduino IDE → téléverser → Moniteur Série à 115200 bauds.

Chaque composant a été testé individuellement afin de garantir son bon fonctionnement avant l'intégration finale.



Le tableau ci-dessous résume les différents tests effectués ainsi que les résultats attendus pour chaque élément du système.

N°	Fichier test	Composant testé	Résultat attendu
1	TEST1_WiFi.ino	Wi-Fi ESP32	IP : 192.168.X.X — signal en dBm
2	TEST2_MPU6050.ino	MPU-6050	AccX/Y/Z qui bougent en secouant
3	TEST3_HCSR04.ino	HC-SR04 ×1	Distances en cm (main = baisse)
4	TEST4_LM393.ino	LM393 + disque	Ticks augmentent (roue tournée)
5	TEST5_Batterie.ino	Pont div. batterie	Tension: ~3.7–4.2V, pct: 0–100%

Tableau 6 : Le test du Composant par Composant

8. Résultats et Conclusion

8.1 Fonctionnalités réalisées

- Détection de mouvement, arrêt et chocs en temps réel (MPU-6050)
- Détection d'obstacles frontaux avec alertes sonores et visuelles (HC-SR04)
- Calcul de position X/Y et distance parcourue — odométrie (LM393 + disque codeur)
- Mesure du niveau de batterie Li-ion 18650 via pont diviseur 110kΩ+200kΩ
- Envoi des données Wi-Fi toutes les secondes (ESP32 → Flask)
- Stockage historique complet dans SQLite
- Tableau de bord web interactif en temps réel (position, distances, alertes, trajet)

8.2 Difficultés rencontrées

- Pont diviseur obligatoire pour protéger les GPIO ESP32 contre les 5V du HC-SR04
- Bruit sur l'ADC de l'ESP32 : moyenne de 10 lectures pour stabiliser la tension batterie
- Organisation du câblage avec de nombreux composants simultanément

8.3 Ce que ce projet nous a appris

- Lecture de capteurs via I2C (MPU-6050) et GPIO (HC-SR04, LM393)
- Circuits de protection : pont diviseur de tension 5V → 3.3V
- Mesure de batterie sans capteur dédié : pont diviseur + ADC
- Programmation serveur web (Flask + SQLite + API REST)
- Interface web temps réel (HTML + CSS + JavaScript)
- Méthodologie IoT : tester composant par composant avant l'intégration

9. Perspectives et Améliorations

- **Détection d'obstacles multi-directionnelle** : ajout de plusieurs capteurs pour détecter les obstacles dans toutes les directions.
- **Câblage amélioré** : optimisation du montage électronique pour obtenir un système plus fiable et organisé.
- **Améliorations logicielles** : amélioration du code et ajout de nouvelles fonctionnalités de surveillance et d'alerte.
- **Autres pistes d'amélioration** : intégration de technologies supplémentaires comme le GPS, la caméra et MQTT.